Challenge 5 — Validazione Contenuto Articolo (Stored XSS)

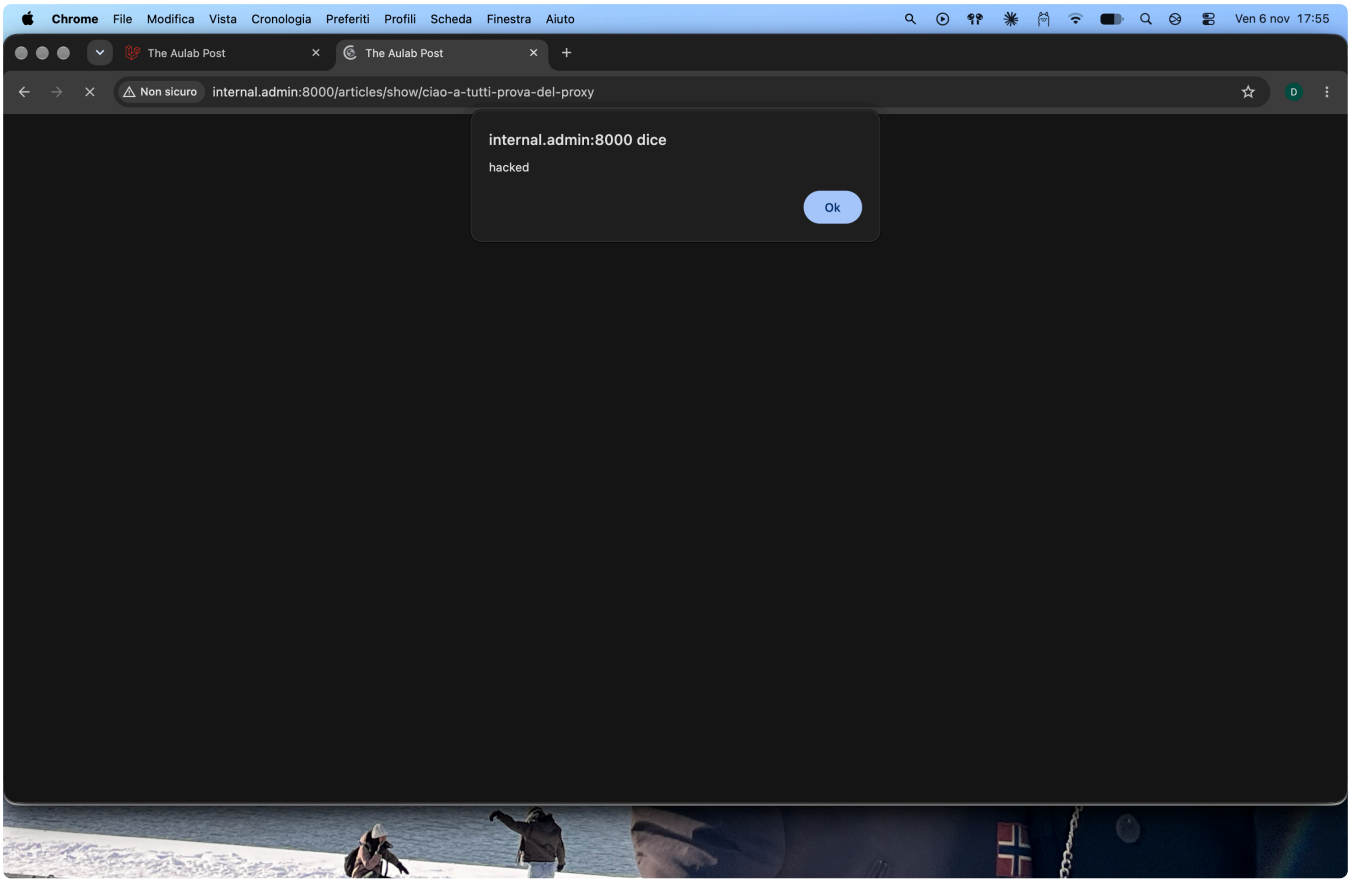
Nov 6, 2026 · @Danilo Noccioli

Attacco

1. BurpSuite configurato in modalità proxy per intercettare il traffico HTTP del browser.
2. Creazione di un articolo con intercettazione attiva sulla richiesta `POST /articles/store`.
3. Modifica del body intercettato, sostituendo il testo con il payload:

```
<script>alert('hacked')</script>
```

4. Richiesta inoltrata (Forward). All'apertura dell'articolo, lo script salvato viene eseguito.



Mitigazione

Principio: non fidarsi mai dell'input, doppia difesa in ingresso e in uscita.

1. Sanitizzazione in input

Il body viene sanificato con **HTMLPurifier** (pacchetto `mews/purifier`) prima del salvataggio, in `ArticleController.php`:

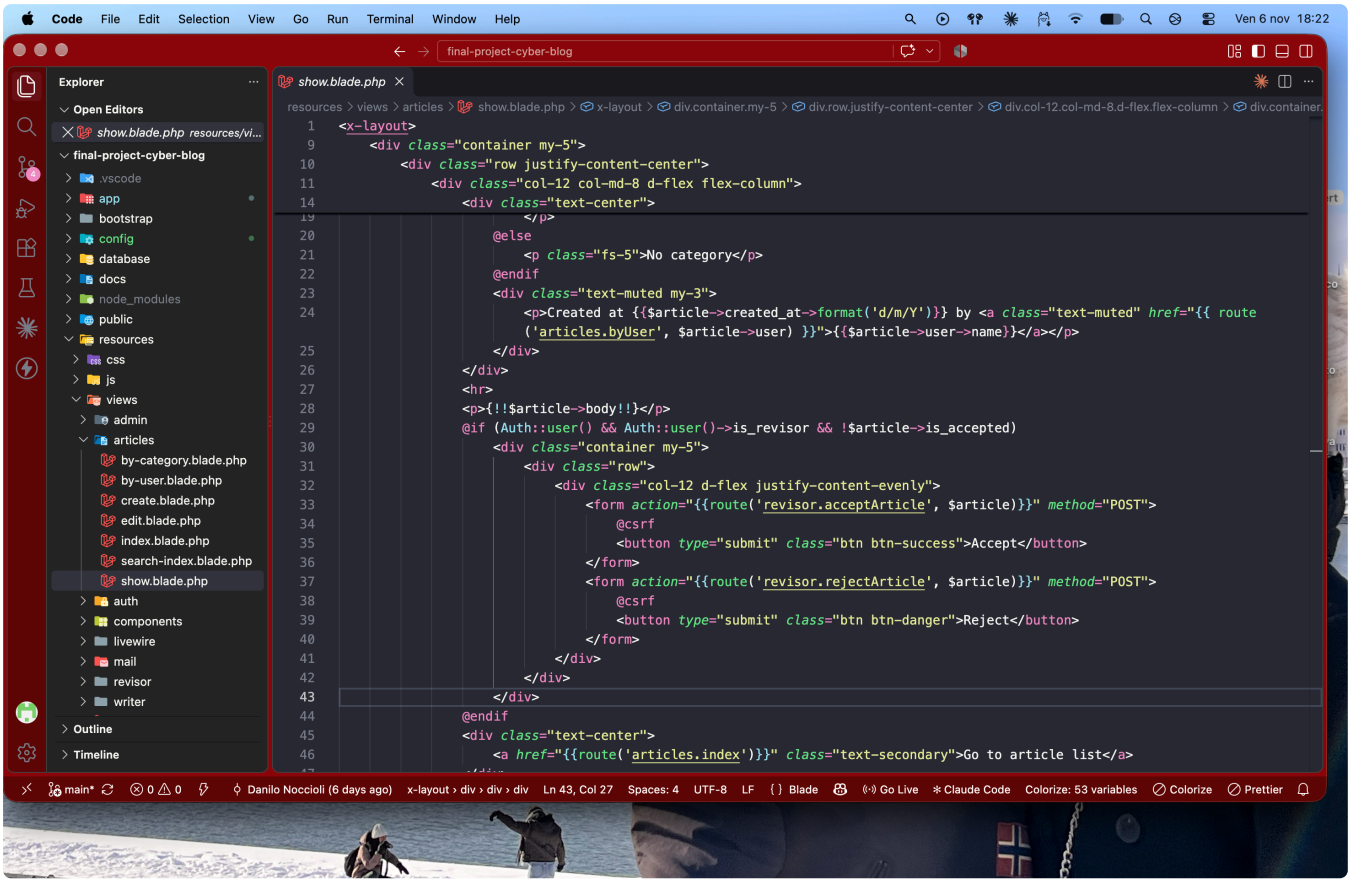
```
use Mews\Purifier\Facades\Purifier;

// in store() e in update()
'body' => Purifier::clean($request->body),
```

La whitelist di default (`config/purifier.php`) consente solo tag sicuri (`div, b, strong, i, em, u, a, ul, ol, li, p, br, span, img`) ed esclude `<script>` e attributi come `onerror`.

2. Escape in fase di visualizzazione

La vista `articles/show.blade.php` stampa il body con `{!! $article->body !!}` (HTML non escapato). Questo resta sicuro solo perché il contenuto arriva già sanificato dal passo 1: l'output si fida dell'input perché l'input è già stato ripulito.



3. Difesa in profondità

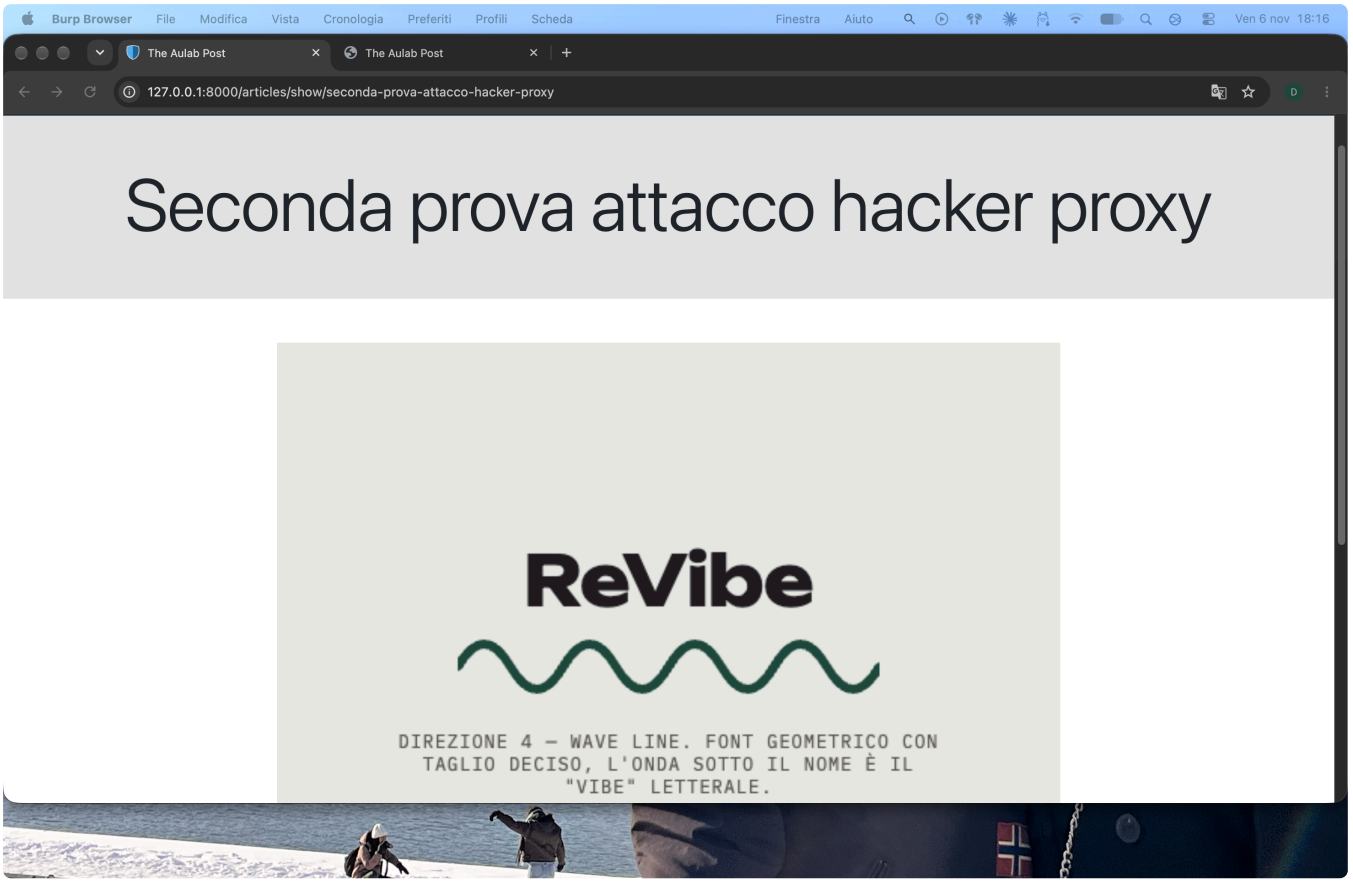
Middleware `SecurityHeaders` (`app/Http/Middleware/SecurityHeaders.php`), registrato globalmente in `bootstrap/app.php`, aggiunge su ogni risposta:

- `Content-Security-Policy`: `script-src 'self'` (+ origine Vite solo in sviluppo), `style-src 'self' 'unsafe-inline'` (compromesso per gli stili inline di Bootstrap/Blade, non legati a input utente), `object-src 'none'`, `frame-src 'none'`, `base-uri 'self'`, `form-action 'self'`
- `X-Frame-Options: DENY` (protezione aggiuntiva da clickjacking)
- `X-Content-Type-Options: nosniff`
- `Referrer-Policy: no-referrer`

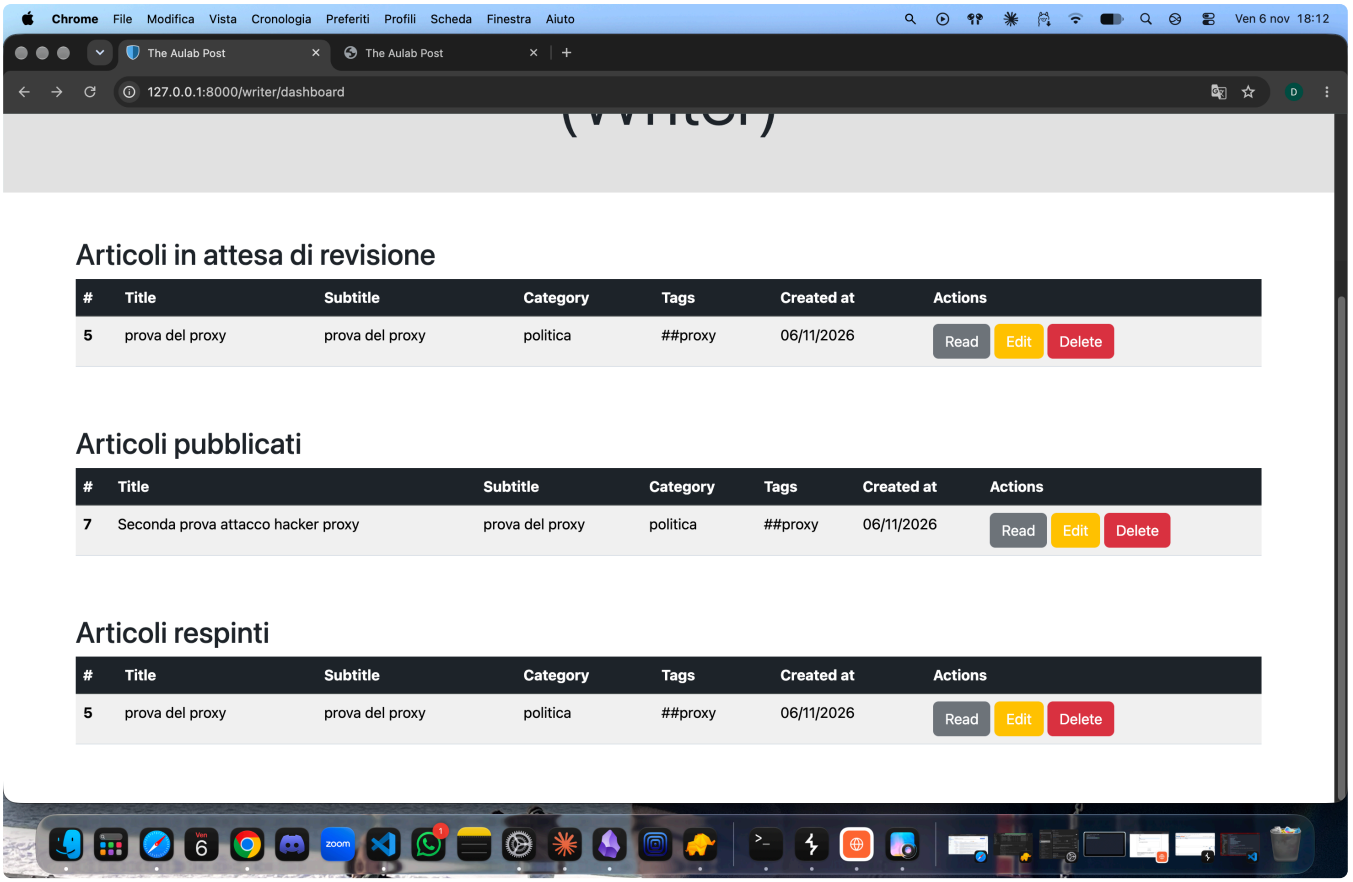
Cookie di sessione `HttpOnly` già attivo (`SESSION_HTTP_ONLY=true`, impostato nella Challenge 2): anche in caso di XSS residuo, lo script non potrebbe leggere il cookie di sessione via `document.cookie`.

Verifica

Rilanciata l'intercettazione con lo stesso payload dopo l'applicazione della mitigazione: il tag `<script>` viene neutralizzato da Purifier (rimosso, non è nella whitelist), nessun alert all'apertura dell'articolo.



La dashboard conferma lo stato degli articoli di test: l'articolo #5 (creato prima della mitigazione, con l'attacco riuscito) risulta tra gli articoli respinti; l'articolo #7 (creato dopo la mitigazione, stesso payload) risulta pubblicato regolarmente senza che lo script si sia attivato.


Chrome File Modifica Vista Cronologia Preferiti Profili Scheda Finestra Aiuto
Ven 6 nov 18:12
The Aulab Post
The Aulab Post
127.0.0.1:8000/writer/dashboard
Articoli in attesa di revisione
Title Subtitle Category Tags Created at Actions
5 prova del proxy prova del proxy politica ##proxy 06/11/2026 Read Edit Delete
Articoli pubblicati
Title Subtitle Category Tags Created at Actions
7 Seconda prova attacco hacker proxy prova del proxy politica ##proxy 06/11/2026 Read Edit Delete
Articoli respinti
Title Subtitle Category Tags Created at Actions
5 prova del proxy prova del proxy politica ##proxy 06/11/2026 Read Edit Delete